200 Java Basics Questions

Master these before touching LeetCode — write every answer yourself, no copy-paste

---

How to use this: Solve in order. For each question, write the code yourself in an IDE or online compiler — don't just think through it mentally. If you get stuck for more than 5-10 minutes on these (they're meant to be easy), look up just the specific syntax you're missing, then close it and finish yourself. Check off each one only after it runs correctly.

1. Variables & Data Types

15 questions — primitive types, casting, constants

- ■ 1. Declare and print an int, double, char, boolean, and String variable.
- ■ 2. Write a program that swaps two integers using a third variable.
- ■ 3. Write a program that swaps two integers without using a third variable.
- ■ 4. Convert an int to a double and print both values.
- ■ 5. Convert a double to an int and observe/print what happens to the decimal part.
- ■ 6. Declare a final (constant) variable and try reassigning it — observe the compiler error.
- ■ 7. Find the size range of int, long, and byte by printing Integer.MAX_VALUE, Long.MAX_VALUE, Byte.MAX_VALUE.
- ■ 8. Write a program that takes user input (Scanner) for name and age, then prints a greeting.
- ■ 9. Declare a char variable and print its underlying int (ASCII) value.
- ■ 10. Convert a char digit ('7') to its integer value (7) without using Character.getNumericValue.
- ■ 11. Write a program demonstrating integer division vs double division (5/2 vs 5.0/2).
- ■ 12. Explain and demonstrate the difference between == and .equals() for two String objects.
- ■ 13. Write a program using var (type inference) for three different data types.
- ■ 14. Declare an array of each primitive type (int[], double[], char[], boolean[]) and print default values.
- ■ 15. Write a program that overflows an int (add 1 to Integer.MAX_VALUE) and print the result.

2. Operators

15 questions — arithmetic, logical, relational, bitwise basics

- ■ 16. Write a program using +, -, *, /, % on two integers and print all results.
- ■ 17. Write a program demonstrating pre-increment vs post-increment (++i vs i++) with printed output.
- ■ 18. Check if a number is even or odd using the modulus operator.
- ■ 19. Write a program using && and || to check if a number is between 10 and 100.
- ■ 20. Write a program demonstrating short-circuit evaluation with && (use a method call that shouldn't run).
- ■ 21. Use the ternary operator to find the maximum of two numbers.
- ■ 22. Use the ternary operator to find the maximum of three numbers.
- ■ 23. Write a program demonstrating compound assignment operators (+=, -=, *=, /=).
- ■ 24. Demonstrate operator precedence: predict then verify the result of 2 + 3 * 4 - 1.
- ■ 25. Write a program using the bitwise AND (&) operator and explain the binary result.
- ■ 26. Write a program using the bitwise OR (|) and XOR (^) operators.
- ■ 27. Write a program using left shift (<<) and right shift (>>) to multiply/divide by powers of 2.
- ■ 28. Check if a number is a power of 2 using bitwise operators.
- ■ 29. Write a program demonstrating the difference between & and && with side-effecting operands.
- ■ 30. Swap two numbers using the XOR operator (no third variable, no arithmetic overflow risk).

3. Control Flow (if/else, switch)

20 questions — conditionals and branching logic

- ■ 31. Check if a number is positive, negative, or zero.
- ■ 32. Check if a year is a leap year.
- ■ 33. Find the largest of three numbers using if-else.
- ■ 34. Check if a triangle is valid given three sides.
- ■ 35. Classify a triangle as equilateral, isosceles, or scalene.
- ■ 36. Write a grading program (marks to letter grade) using if-else ladder.
- ■ 37. Rewrite the grading program using switch-case.
- ■ 38. Check if a character is a vowel or consonant.
- ■ 39. Check if a character is an uppercase letter, lowercase letter, digit, or special character.
- ■ 40. Write a simple calculator (+, -, *, /) using switch-case based on operator input.
- ■ 41. Check if a number is divisible by both 3 and 5.
- ■ 42. Determine the quadrant of a point (x, y) on a Cartesian plane.
- ■ 43. Write a program to check if three given lengths can form a right triangle.
- ■ 44. Check if a given day number (1-7) is a weekday or weekend using switch.
- ■ 45. Write nested if-else to categorize BMI (underweight/normal/overweight/obese) given height and weight.
- ■ 46. Check if a number is a palindrome (e.g. 121) using conditionals and math (no string conversion).
- ■ 47. Write a program that determines the season given a month number.
- ■ 48. Use a switch expression (Java 14+ style, arrow syntax) to rewrite one of the above.
- ■ 49. Check if a given character is present in "aeiouAEIOU" using conditionals (no loops).
- ■ 50. Write a program that validates a simple password (length >= 8, has a digit) using if-else.

4. Loops (for, while, nested)

25 questions — the backbone of most DSA problems

- ■ 51. Print numbers from 1 to N using a for loop.
- ■ 52. Print numbers from N to 1 using a while loop.
- ■ 53. Print all even numbers between 1 and 100.
- ■ 54. Calculate the sum of numbers from 1 to N.
- ■ 55. Calculate the factorial of a number using a loop (not recursion).
- ■ 56. Print the multiplication table of a given number.
- ■ 57. Check if a number is prime using a loop.
- ■ 58. Print all prime numbers between 1 and 100.
- ■ 59. Reverse the digits of a number using a loop (e.g. 123 -> 321).
- ■ 60. Check if a number is a palindrome using a loop (e.g. 121).
- ■ 61. Find the sum of digits of a number.
- ■ 62. Count the number of digits in a number.
- ■ 63. Find the GCD of two numbers using a loop (Euclidean or brute force).
- ■ 64. Find the LCM of two numbers.
- ■ 65. Print the Fibonacci series up to N terms using a loop.
- ■ 66. Print a right-angled triangle pattern of stars using nested loops.
- ■ 67. Print a pyramid (centered triangle) pattern of stars using nested loops.
- ■ 68. Print a number pattern where each row shows 1, 12, 123, 1234, etc.
- ■ 69. Print an inverted right-angled triangle pattern.
- ■ 70. Print a diamond pattern of stars using nested loops.
- ■ 71. Find the sum of all even numbers between 1 and N using continue to skip odd numbers.
- ■ 72. Write a program that uses break to exit a loop once a target value is found in a sequence.
- ■ 73. Simulate a do-while loop for a menu that repeats until the user enters 'exit'.
- ■ 74. Check if a number is an Armstrong number (e.g. 153 = 1^3+5^3+3^3).
- ■ 75. Print all Armstrong numbers between 1 and 1000.

5. Arrays (1D & 2D)

30 questions — the single most important category for DSA

- ■ 76. Declare, initialize, and print all elements of an int array.
- ■ 77. Find the sum of all elements in an array.
- ■ 78. Find the maximum and minimum element in an array (without using built-in max/min).
- ■ 79. Find the second largest element in an array without sorting.
- ■ 80. Reverse an array in place (without creating a new array).
- ■ 81. Count how many even and odd numbers are in an array.
- ■ 82. Find the average of elements in an array.
- ■ 83. Check if an array is sorted in ascending order.
- ■ 84. Search for a target value in an array using linear search.
- ■ 85. Implement binary search on a sorted array (iterative, not using library functions).
- ■ 86. Remove duplicate elements from an array (print the unique elements).
- ■ 87. Find the frequency of each element in an array (print counts).
- ■ 88. Merge two sorted arrays into a single sorted array.
- ■ 89. Find the intersection of two arrays (common elements).
- ■ 90. Find the union of two arrays (all unique elements from both).
- ■ 91. Rotate an array to the left by one position.
- ■ 92. Rotate an array to the left by k positions.
- ■ 93. Find all pairs in an array that sum to a target value (brute force, nested loops).
- ■ 94. Find the missing number in an array containing 1 to N with one number missing.
- ■ 95. Move all zeros in an array to the end while keeping other elements in order.
- ■ 96. Find the index of the first and last occurrence of a target value.
- ■ 97. Sort an array using bubble sort (write it yourself, don't use Arrays.sort()).
- ■ 98. Sort an array using selection sort.
- ■ 99. Sort an array using insertion sort.
- ■ 100. Declare and initialize a 2D array (matrix) and print all elements row by row.
- ■ 101. Find the sum of all elements in a 2D array.
- ■ 102. Find the sum of each row and each column in a 2D matrix, printed separately.
- ■ 103. Find the transpose of a matrix.
- ■ 104. Check if a matrix is symmetric (equal to its transpose).
- ■ 105. Print the diagonal elements of a square matrix (both main and anti-diagonal).

6. Strings

30 questions — core string manipulation used constantly in DSA

- ■ 106. Find the length of a string without using .length().
- ■ 107. Reverse a string without using StringBuilder.reverse().
- ■ 108. Check if a string is a palindrome.
- ■ 109. Count the number of vowels and consonants in a string.
- ■ 110. Count the frequency of each character in a string using a HashMap.
- ■ 111. Check if two strings are anagrams of each other.
- ■ 112. Find the first non-repeating character in a string.
- ■ 113. Remove all whitespace from a string.
- ■ 114. Convert a string to uppercase and lowercase without using built-in methods (manual char manipulation).
- ■ 115. Check if a string contains only digits.
- ■ 116. Check if a string contains only alphabetic characters.
- ■ 117. Count the number of words in a sentence (split on whitespace).
- ■ 118. Reverse the order of words in a sentence (not the letters).
- ■ 119. Check if two strings are rotations of each other (e.g. "abcd" and "cdab").
- ■ 120. Find the longest word in a sentence.

■ 121. Remove duplicate characters from a string, keeping first occurrence order.
■ 122. Convert a String to a char array and print each character.
■ 123. Check if a string starts with or ends with a specific substring, manually (no startsWith/endsWith).
■ 124. Find all substrings of a given string.
■ 125. Find the index of the first occurrence of a substring, manually (no indexOf).
■ 126. Count the number of occurrences of a specific character in a string.
■ 127. Capitalize the first letter of every word in a sentence.
■ 128. Check if a string has all unique characters (no repeats).
■ 129. Compress a string using counts (e.g. "aaabb" -> "a3b2").
■ 130. Convert an integer to a String without using String.valueOf() or Integer.toString().
■ 131. Convert a String of digits to an integer without using Integer.parseInt().
■ 132. Build a String using StringBuilder by appending characters in a loop, then print it.
■ 133. Check if a string is a valid palindrome ignoring spaces and case (e.g. "A man a plan a canal Panama").
■ 134. Find the longest common prefix among an array of strings.
■ 135. Split a comma-separated string into individual values and print each on a new line.

7. Methods

15 questions — writing your own functions cleanly

■ 136. Write a method that takes two ints and returns their sum.
■ 137. Write a method that checks if a number is prime and returns a boolean.
■ 138. Write a method that returns the factorial of a number.
■ 139. Write a method that takes an array and returns the max element.
■ 140. Write a method with a default-like behavior using method overloading (same name, different params).
■ 141. Write a method that returns multiple values by returning an array.
■ 142. Write a method that takes a String and returns its reverse.
■ 143. Write a void method that prints a pattern, called from main with a parameter for size.
■ 144. Write a method that takes an array and modifies it in place (e.g. doubles every element) — observe pass-by-reference behavior.
■ 145. Write a method that takes a primitive int and tries to modify it — observe pass-by-value behavior.
■ 146. Write two overloaded methods: one takes two ints, another takes two doubles, both named add.
■ 147. Write a recursive-looking method signature but implement it iteratively first (helper method practice).
■ 148. Write a method isPalindrome(String s) and call it from three different test cases in main.
■ 149. Write a method that takes a 2D array and returns the sum of all elements.
■ 150. Write a method with a variable number of arguments (varargs) that returns their sum.

8. Recursion

15 questions — draw the call stack by hand for each one

■ 151. Write a recursive method to calculate factorial of a number.
■ 152. Write a recursive method to calculate the nth Fibonacci number.
■ 153. Write a recursive method to calculate the sum of digits of a number.
■ 154. Write a recursive method to reverse a string.
■ 155. Write a recursive method to check if a string is a palindrome.
■ 156. Write a recursive method to find the sum of elements in an array.
■ 157. Write a recursive method to find the maximum element in an array.
■ 158. Write a recursive method to calculate x raised to the power n.
■ 159. Write a recursive method to print numbers from 1 to N.
■ 160. Write a recursive method to print numbers from N to 1.
■ 161. Write a recursive method to calculate GCD of two numbers.

■ 162. Write a recursive method to count the number of digits in a number.
■ 163. Write a recursive method that converts a decimal number to binary (as a String).
■ 164. Write a recursive method to check if an array is sorted.
■ 165. Draw the full call stack (on paper, then describe in comments) for fibonacci(5), showing every call and return.

9. Collections Basics (ArrayList, HashMap, HashSet)

20 questions — you'll use these constantly in DSA solutions

■ 166. Create an ArrayList of integers, add 5 elements, and print it.
■ 167. Remove an element from an ArrayList by value and by index.
■ 168. Iterate over an ArrayList using a for loop, enhanced for loop, and an Iterator.
■ 169. Check if an ArrayList contains a specific element.
■ 170. Sort an ArrayList of integers using Collections.sort().
■ 171. Convert an int array to an ArrayList and back.
■ 172. Create an ArrayList of Strings and find the longest string in it.
■ 173. Create a HashMap<String, Integer> and use it to count word frequency in a sentence.
■ 174. Iterate over a HashMap's keys, values, and entries separately, printing each.
■ 175. Check if a key exists in a HashMap before accessing it (getOrDefault vs containsKey).
■ 176. Find the key with the maximum value in a HashMap.
■ 177. Create a HashSet and use it to find duplicate elements in an array.
■ 178. Use a HashSet to check if two arrays share any common elements.
■ 179. Create a 2D ArrayList (ArrayList of ArrayLists) and populate it with values.
■ 180. Use a HashMap to group words by their length from a list of words.
■ 181. Use a HashMap to find the first non-repeating character in a string (revisit Q4 from Strings, using HashMap this time).
■ 182. Create a LinkedList, add elements, and demonstrate addFirst/addLast/removeFirst/removeLast.
■ 183. Use a Stack (java.util.Stack) to reverse a string.
■ 184. Use a Queue (LinkedList as Queue) to simulate a simple FIFO print queue.
■ 185. Compare the time to search for an element in an ArrayList vs a HashSet (write both, comment on why one is faster).

10. Basic OOP (Classes & Objects)

15 questions — needed to read/write realistic Java code

■ 186. Create a class Student with fields name and age, and a method to print details.
■ 187. Create a constructor for the Student class and use it to initialize objects.
■ 188. Create multiple Student objects in main and print each one's details.
■ 189. Add a method to Student that returns true if the student is an adult (age >= 18).
■ 190. Create a class Rectangle with width and height fields, and methods to calculate area and perimeter.
■ 191. Demonstrate method overloading inside a class (e.g. two constructors with different parameters).
■ 192. Create a class with a static method and call it without creating an object.
■ 193. Create a class with a static counter field that increments every time a new object is created.
■ 194. Create a simple class hierarchy: Animal (parent) and Dog (child) using extends, with a method overridden in Dog.
■ 195. Demonstrate the use of 'this' keyword to distinguish instance fields from constructor parameters.
■ 196. Create a class with private fields and public getter/setter methods (encapsulation).
■ 197. Create an array of objects (e.g. Student[]) and iterate through it printing each object's data.
■ 198. Create a class that implements toString() to customize how an object prints.
■ 199. Create two classes where one class's method takes an object of the other class as a parameter.

■ 200. Write a simple class representing a BankAccount with deposit() and withdraw() methods that update a balance field.

Progress Tracking

Track your pace here. Aim to clear all 200 in 7-10 focused days (20-30/day). Do not move to LeetCode until every box above is checked and you can redo at least 5 random ones cold, without looking back at your old code.

Category	Count	Target Day	Done
1. Variables & Data Types	15	Day 1	
2. Operators	15	Day 1	
3. Control Flow	20	Day 2	
4. Loops	25	Day 3	
5. Arrays	30	Day 4-5	
6. Strings	30	Day 5-6	
7. Methods	15	Day 6	
8. Recursion	15	Day 7	
9. Collections Basics	20	Day 8	
10. Basic OOP	15	Day 9	

Readiness check: Once done, try this cold: "Write a Java method that takes an array of integers and returns true if any two numbers in the array sum to a target value." If you can produce a working solution yourself, you're ready to start LeetCode patterns.